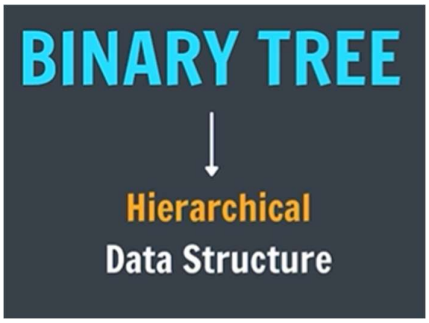
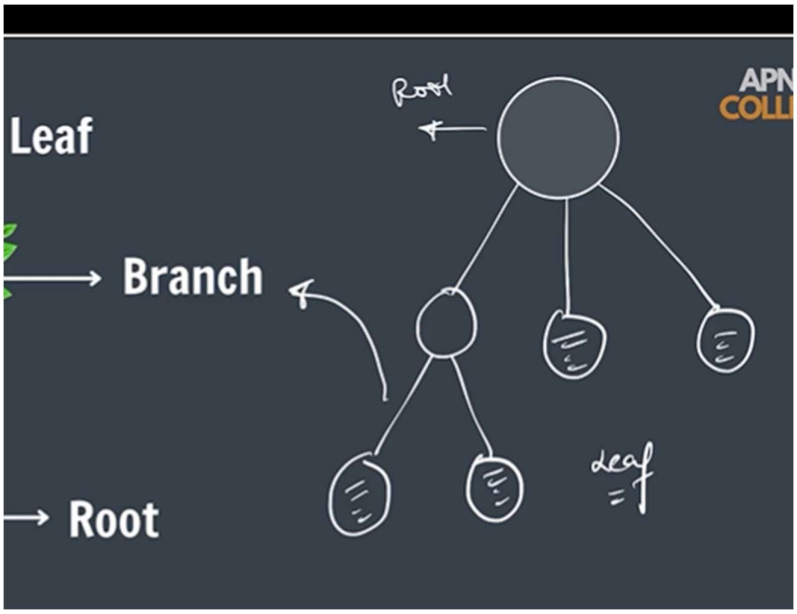
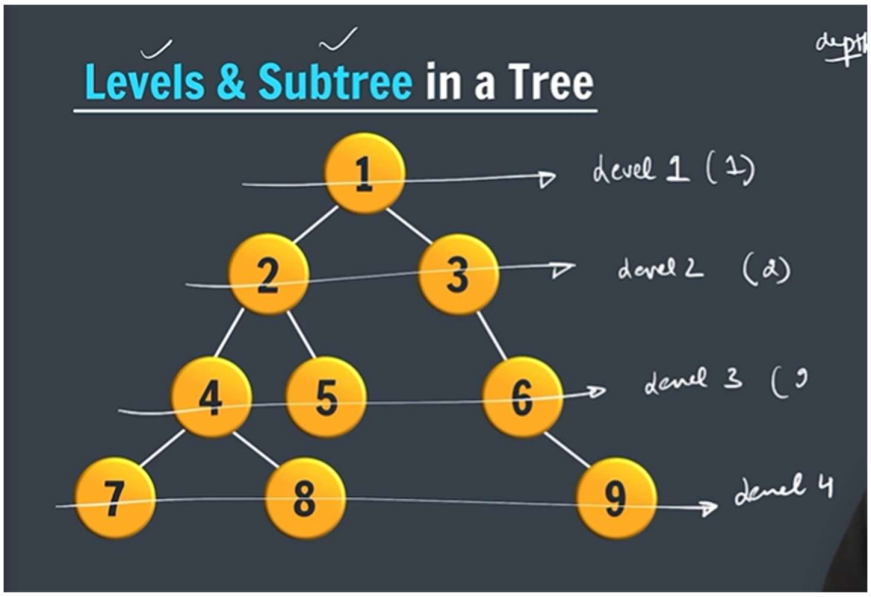
→Binary Tree

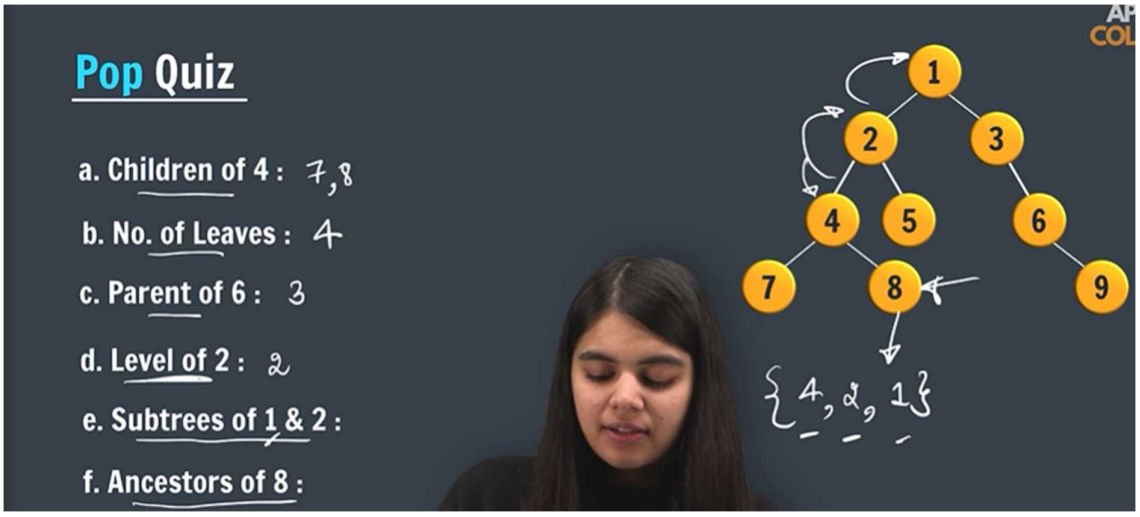


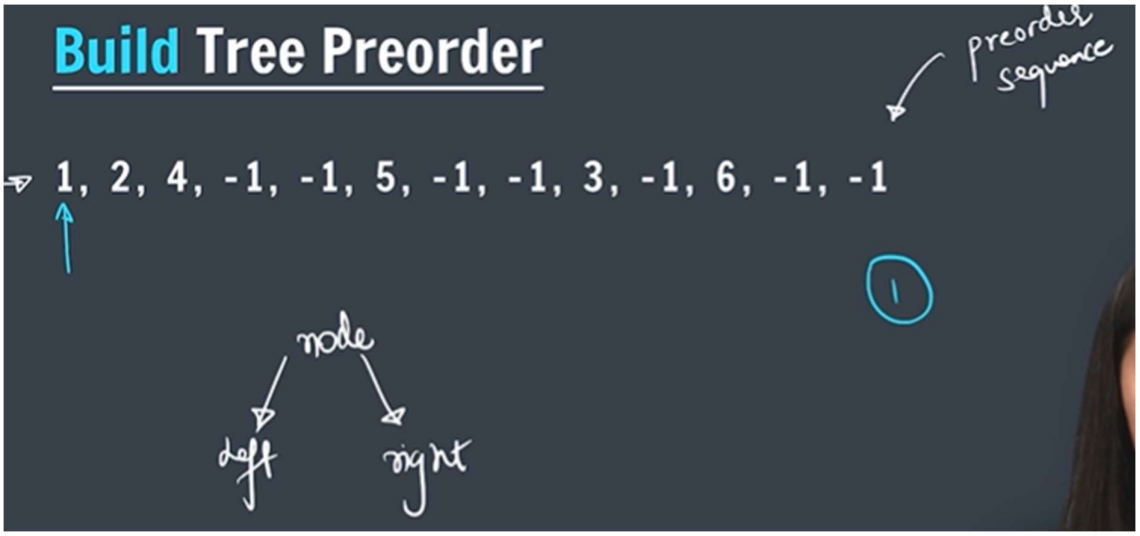

→binary tree atmost 2 children

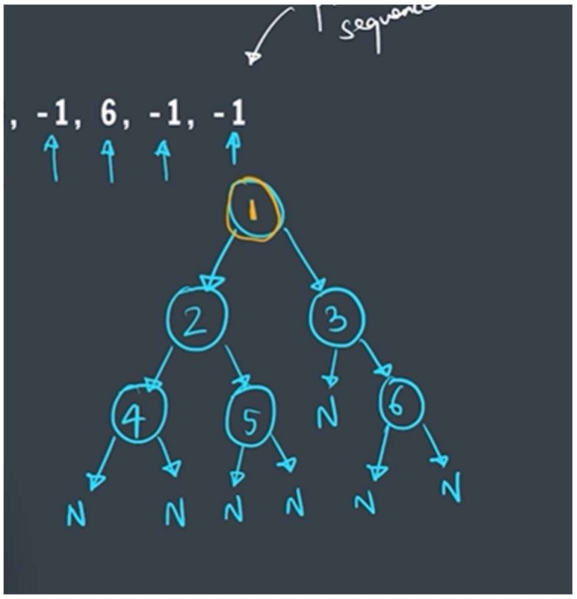
→Level and subtree in a Tree



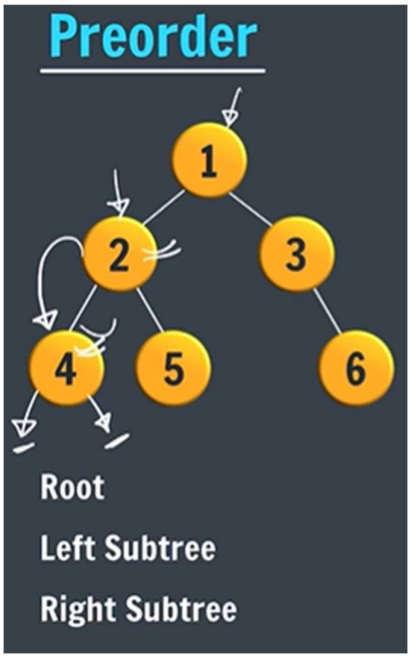



→**Build Tree Preorder**



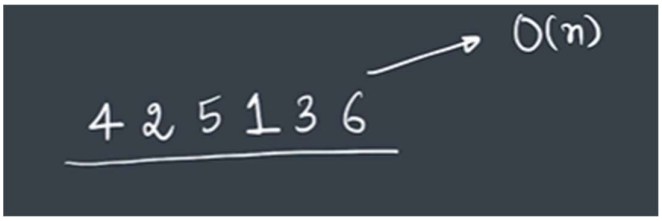



→Preorder Traversal

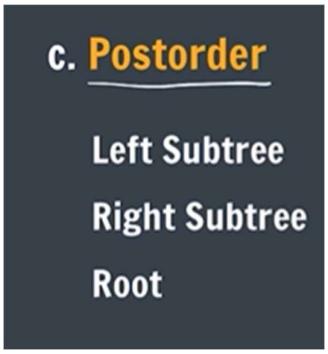




→Inorder Traversal

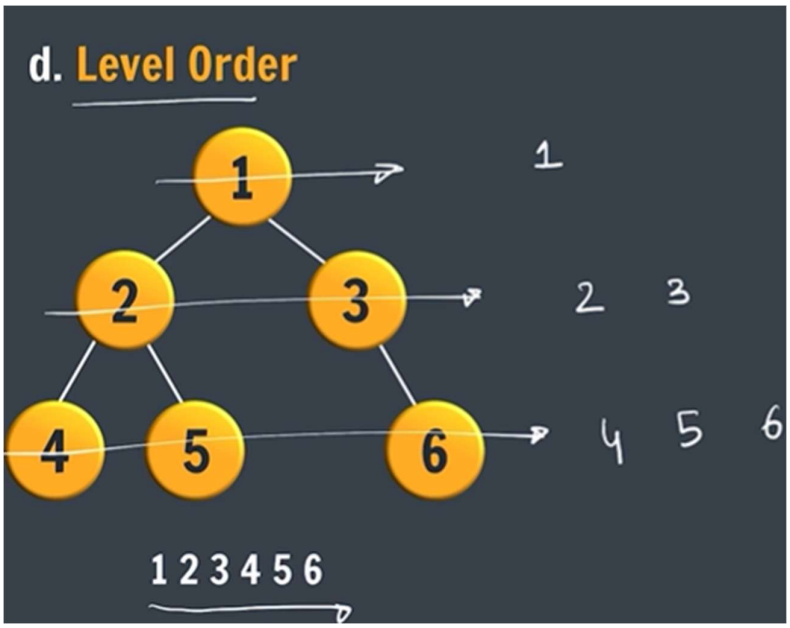



→Postorder Traversal





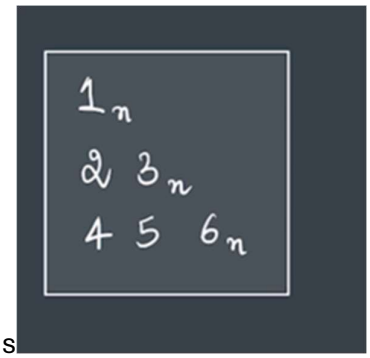

→Level Order




->in level order we add the one node in queue and we add the left and right nodesof that node and remove the root node

->if we do like this we get like this 1,2,3,4,5,6

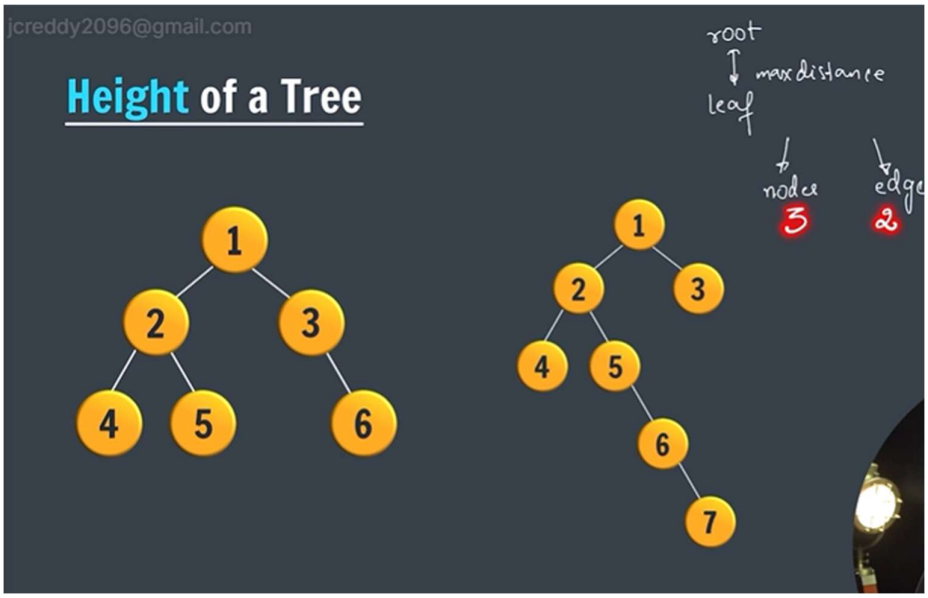
But we want like this



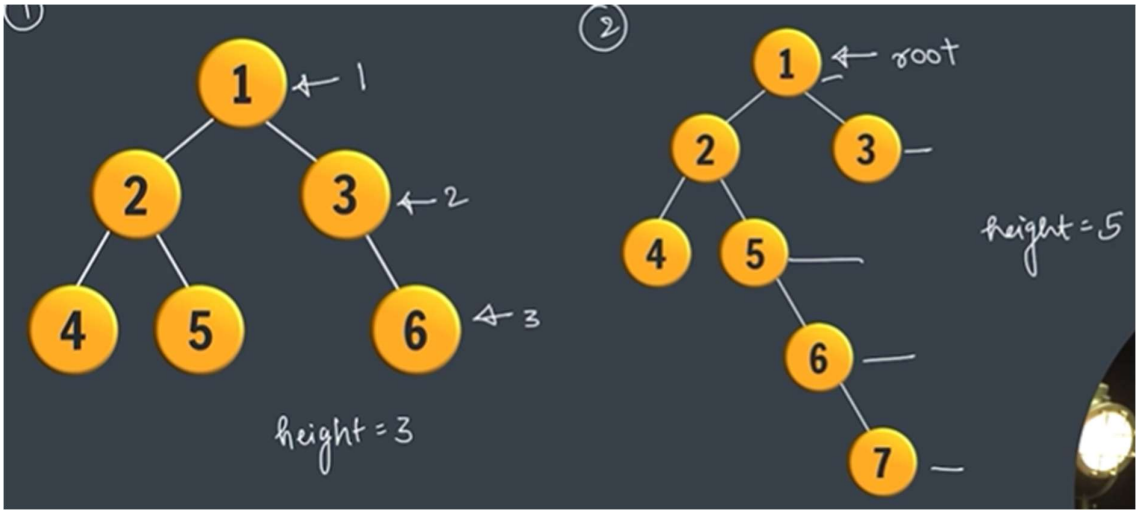
→for that we add first add root and null in the queue and after adding the left and right nodes we remove the root node and then after adding other nodes we remove the null and then again we add the null in the queue until end

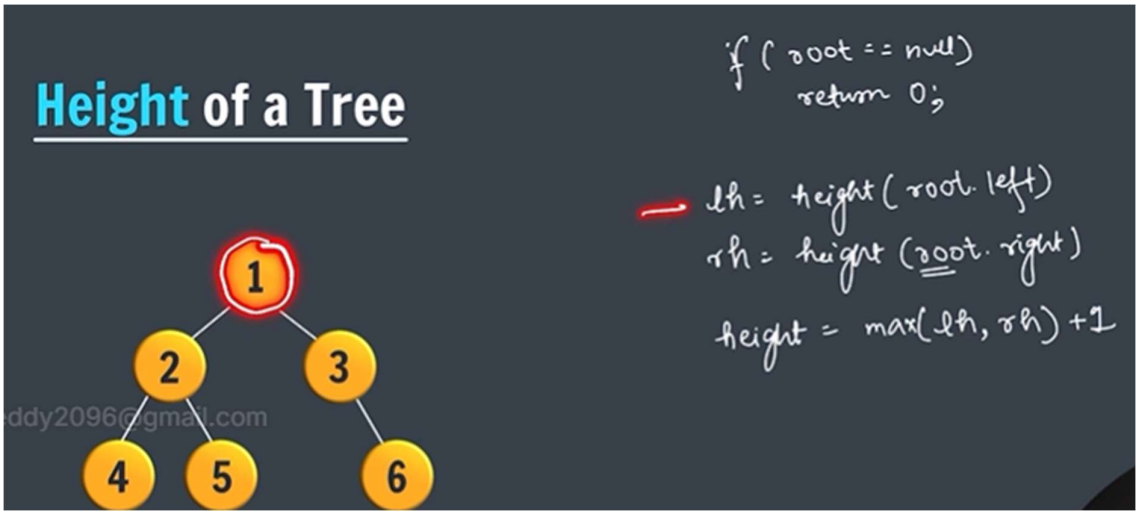
→for this time complexity is O(n).

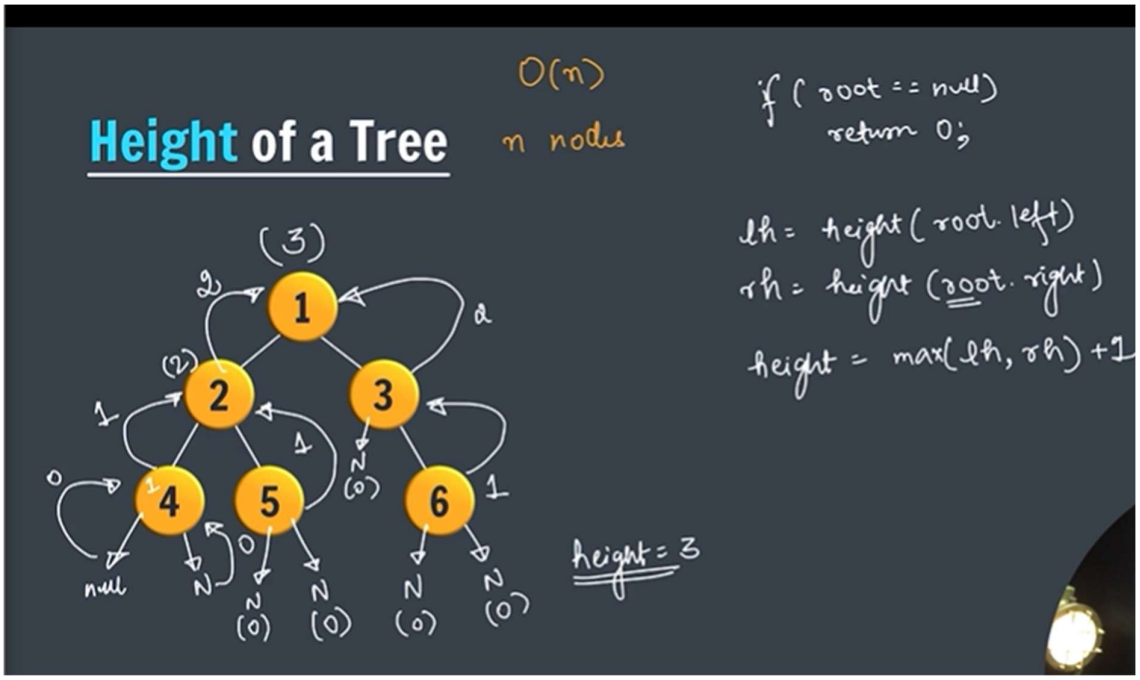
→Height of a Tree



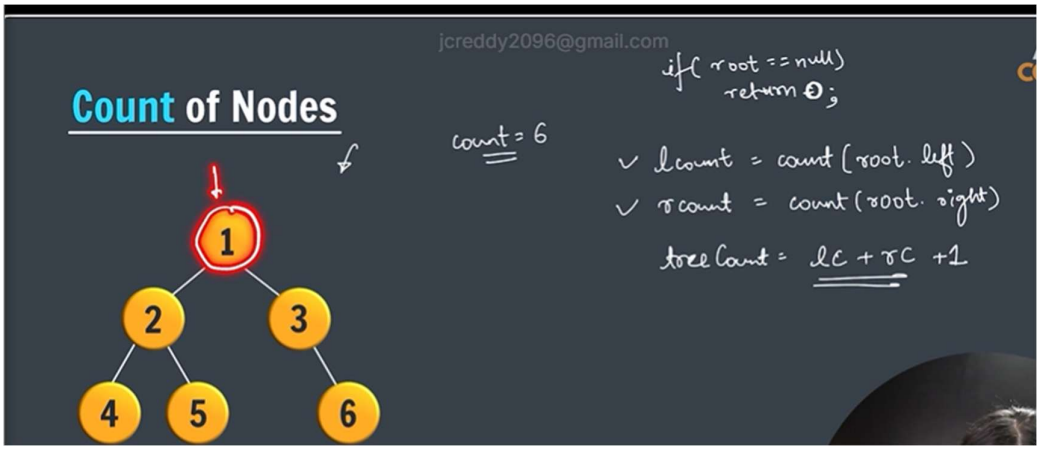

→we take the deepest node to calculate the height of the tree

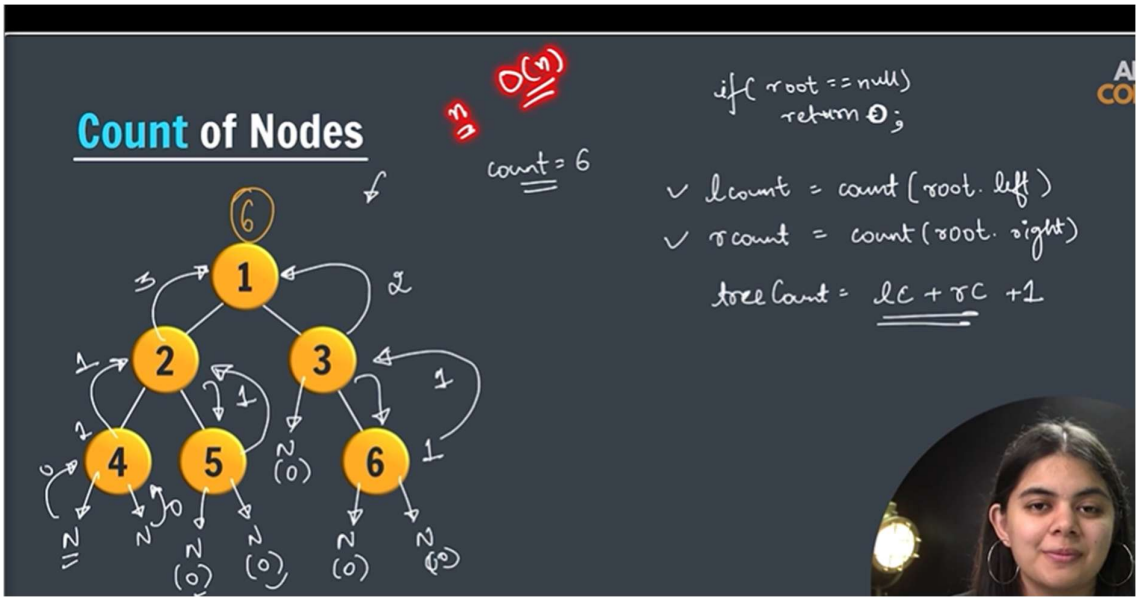




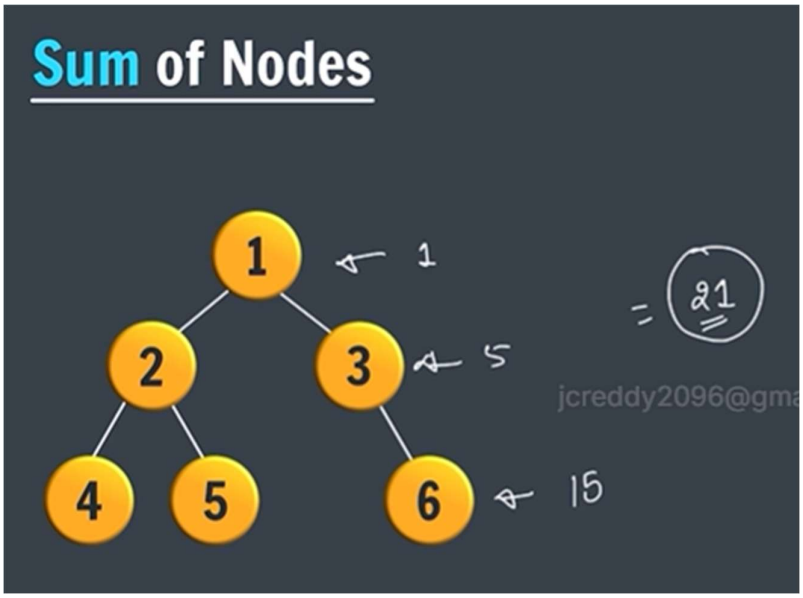



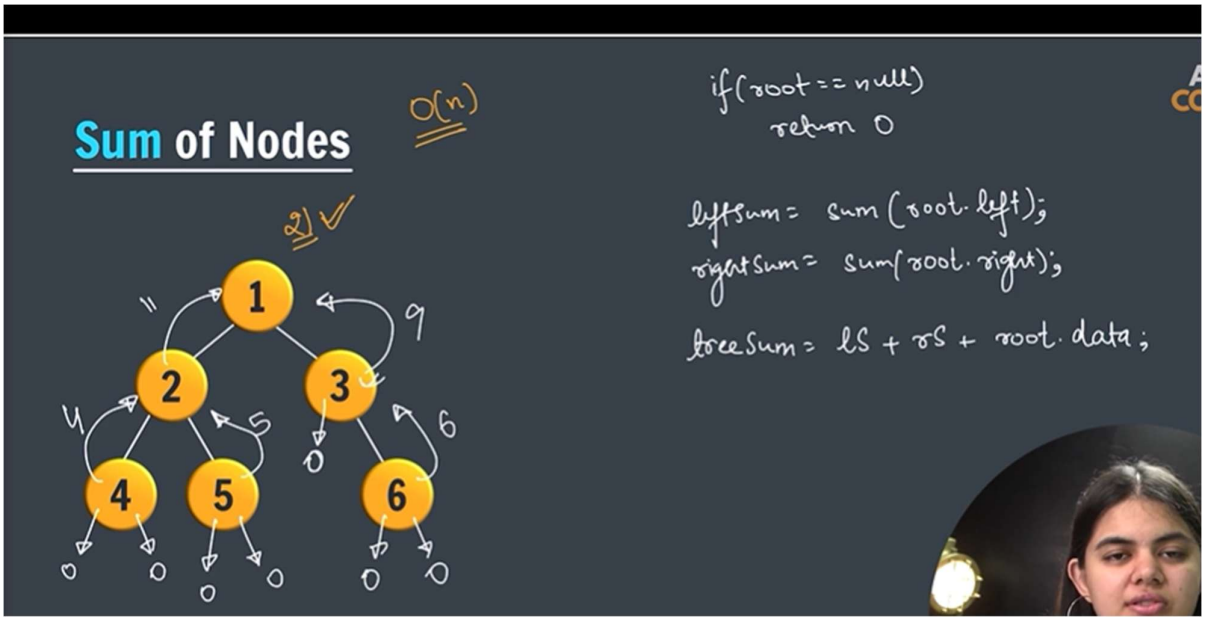

→Count of Nodes




Count of Nodes
O(n)
n = 6
count = 6
if( root == null)
return 0;
lcount = count(root.left)
rcount = count(root.right)
treeCount = lc + rc + 1

→sum of nodes

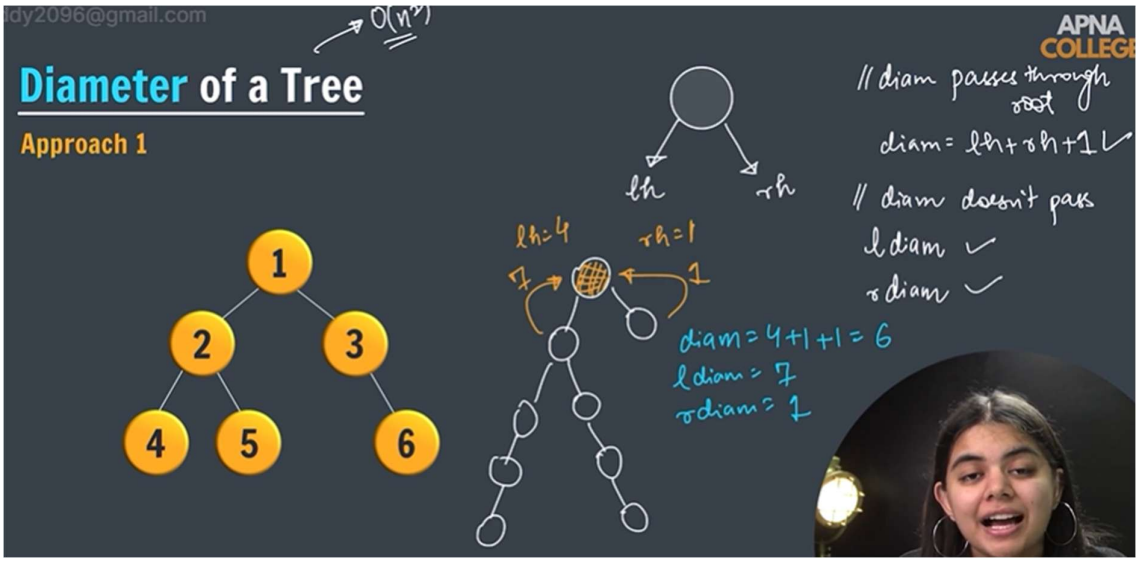

Sum of Nodes
1
2
3
4
5
6
1
5
15
= 21

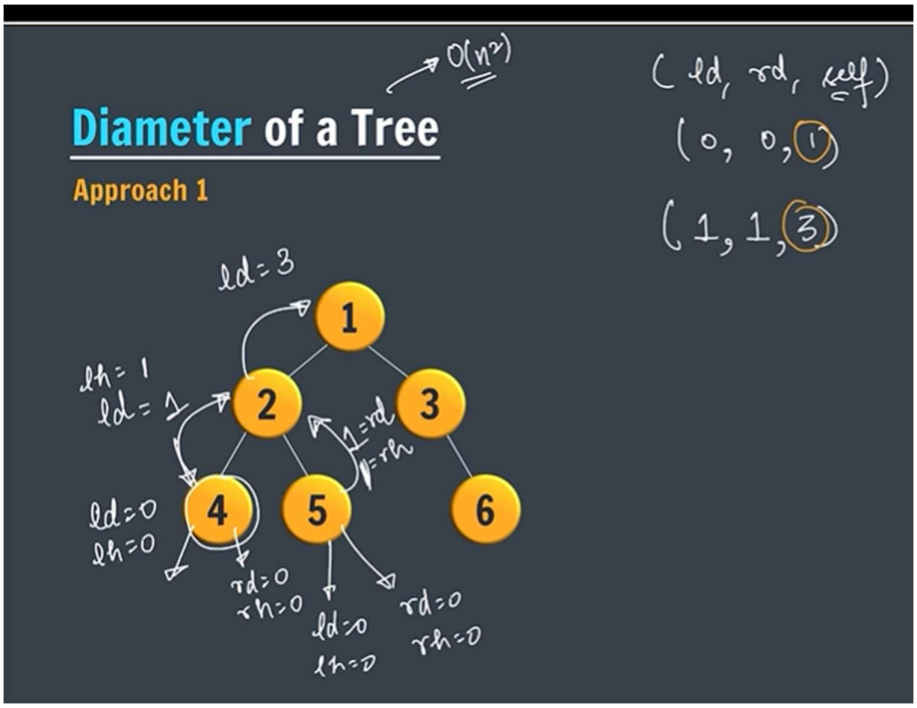


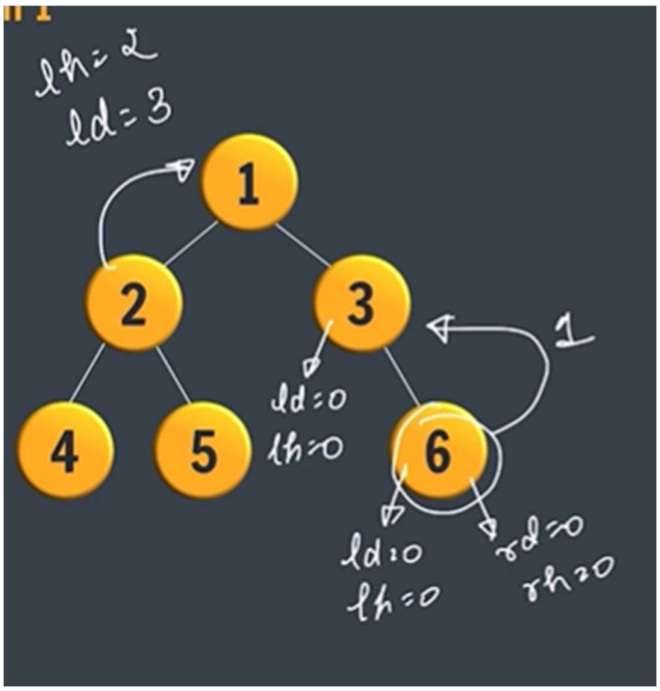

→Binary tree-2

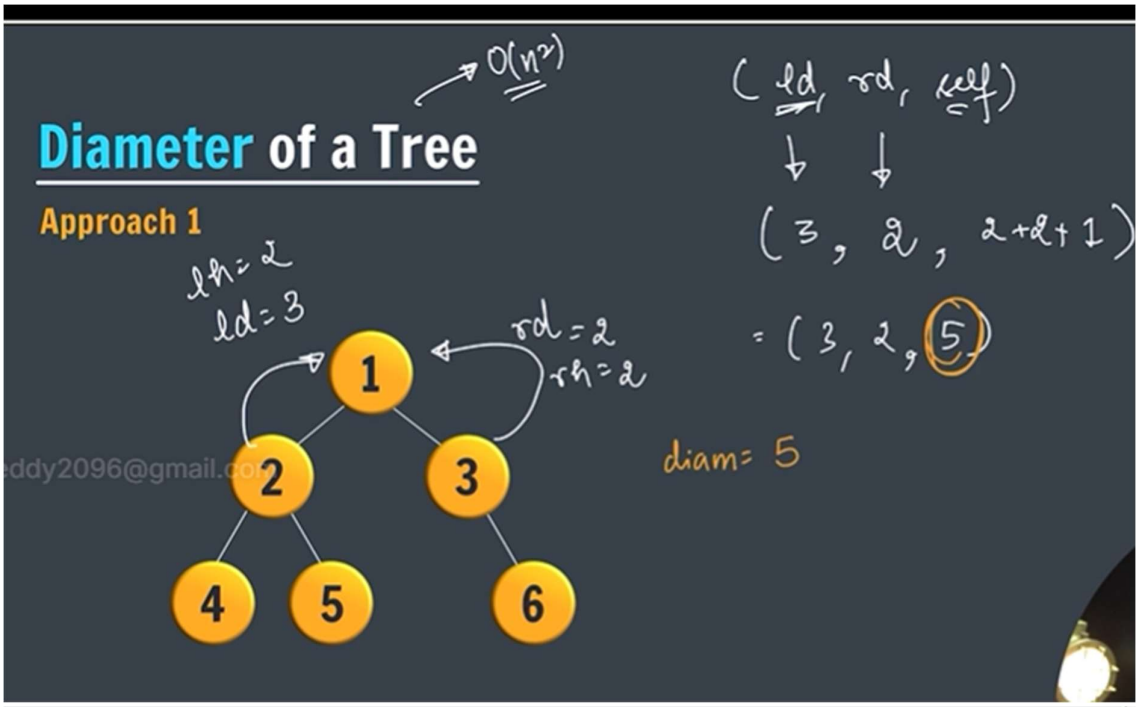
→Diameter of a tree:

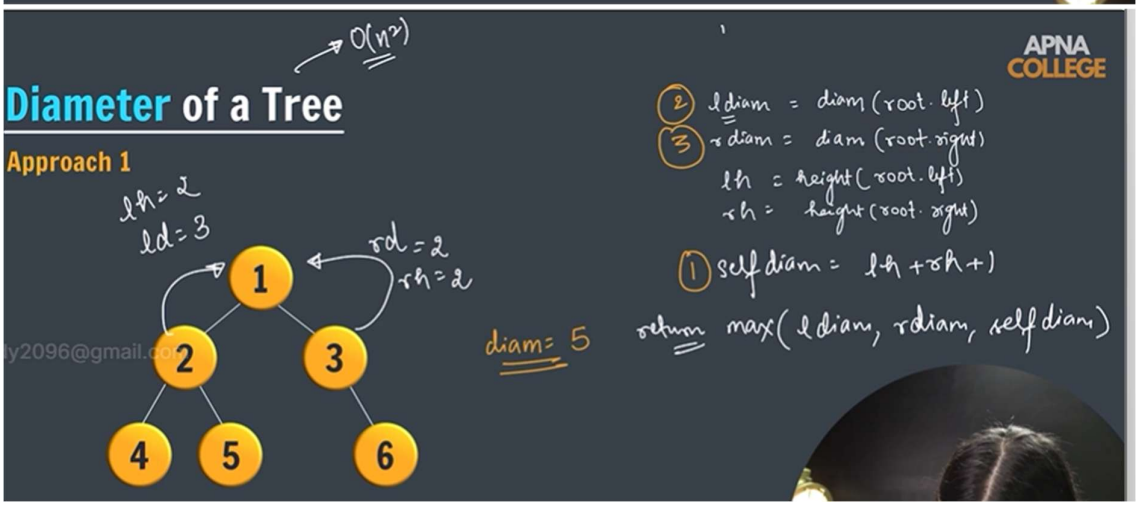
No. of nodes in the longest path between 2 leaves



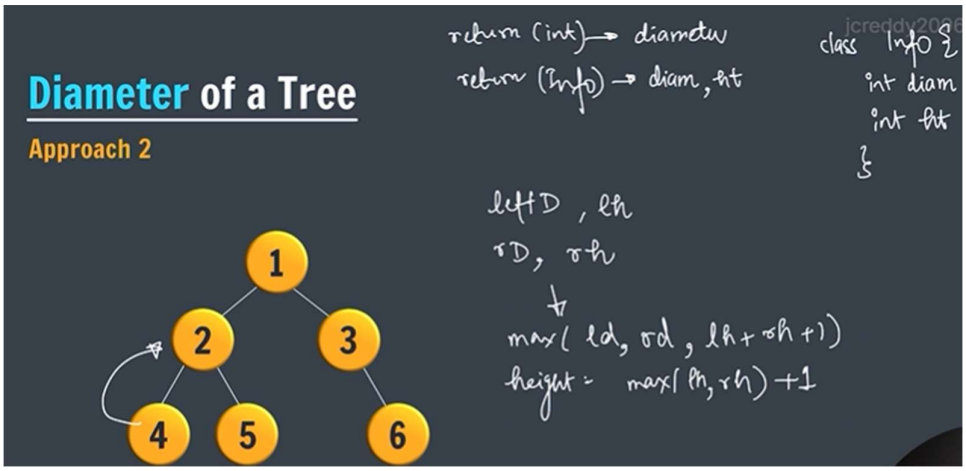

O(n²)
Diameter of a Tree
Approach 1
(ld, rd, self)
(0, 0, 1)
(1, 1, 3)
ld=3
lh=1
ld=1
1=rd
1=rh
ld=0
lh=0
rd=0
rh=0
ld=0
lh=0
rd=0
rh=0
1
2
3
4
5
6

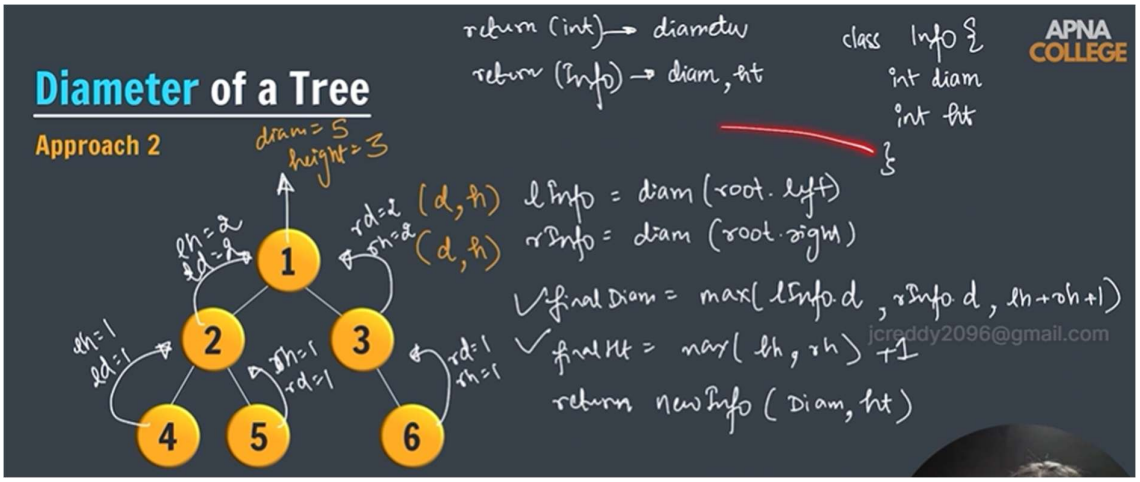

lh=2
ld=3
1
2
3
4
5
6
1
ld=0
lh=0
ld=0
lh=0
rd=0
rh=0


O(n²)
Diameter of a Tree
Approach 1
lh = 2
ld = 3
rd = 2
rh = 2
1
2
3
4
5
6
diam = 5
(ld, rd, self)
(3, 2, 2+2+1)
= (3, 2, 5)

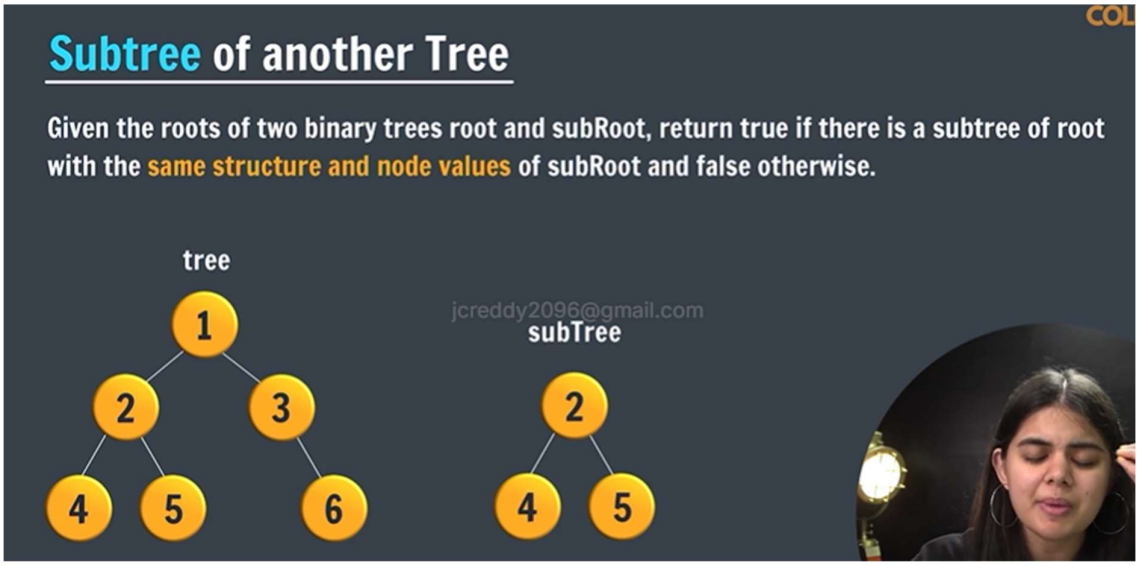

O(n²)
Diameter of a Tree
Approach 1
APNA COLLEGE
lh = 2
ld = 3
rd = 2
rh = 2
1
2
3
4
5
6
diam = 5
② ldiam = diam(root.left)
③ rdiam = diam(root.right)
lh = height(root.left)
rh = height(root.right)
① selfdiam = lh + rh + 1
return max(ldiam, rdiam, selfdiam)

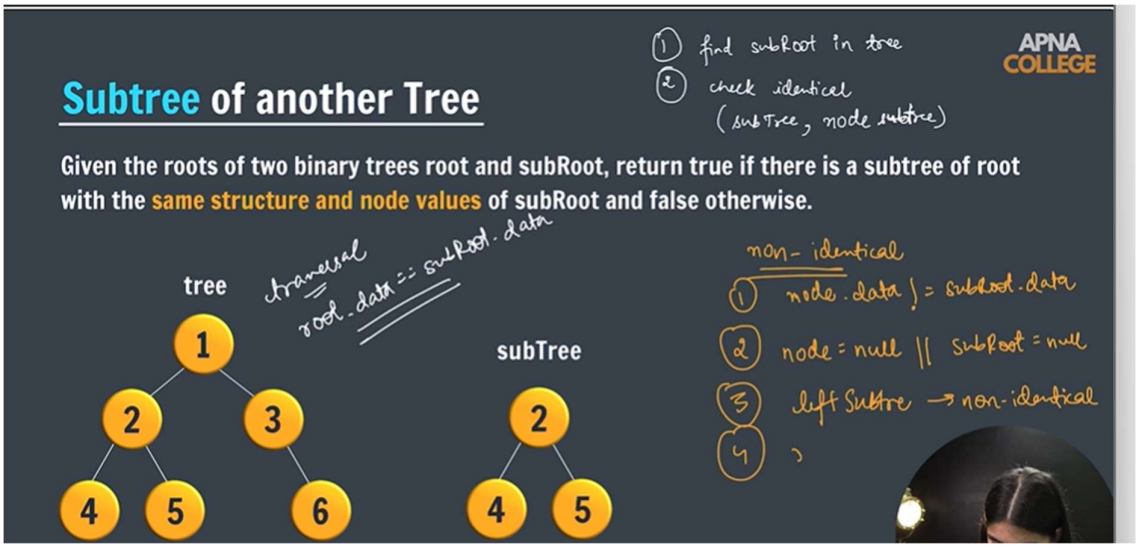
→Approach-2

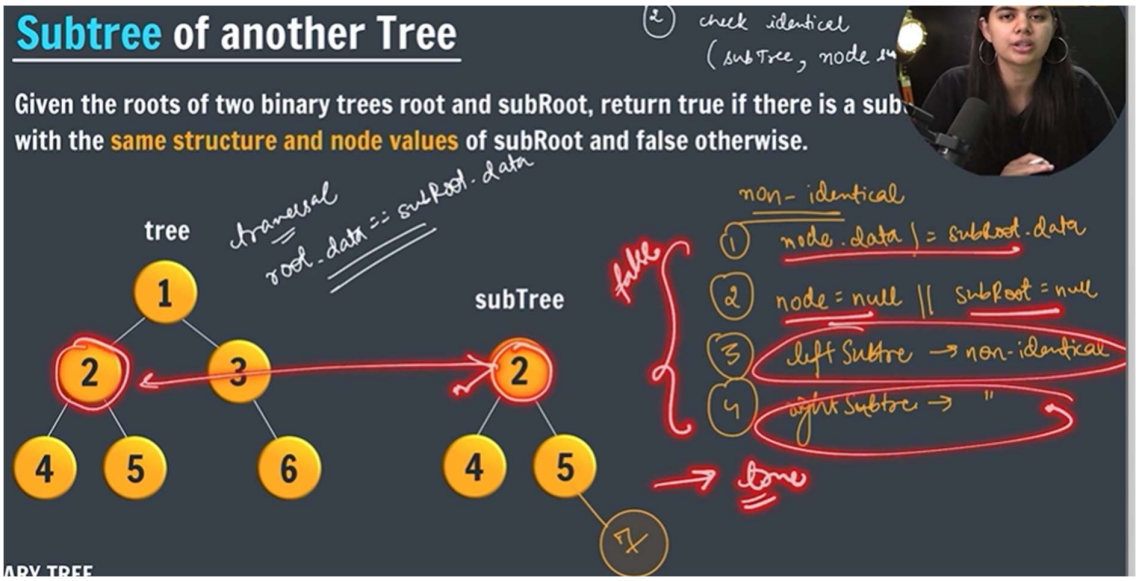

Diameter of a Tree
Approach 2
return (int) → diameter
return (Info) → diam, ht
class Info {
int diam
int ht
}
leftD, lh
rD, rh
max(ld, rd, lh + rh + 1)
height = max(lh, rh) + 1
1
2
3
4
5
6



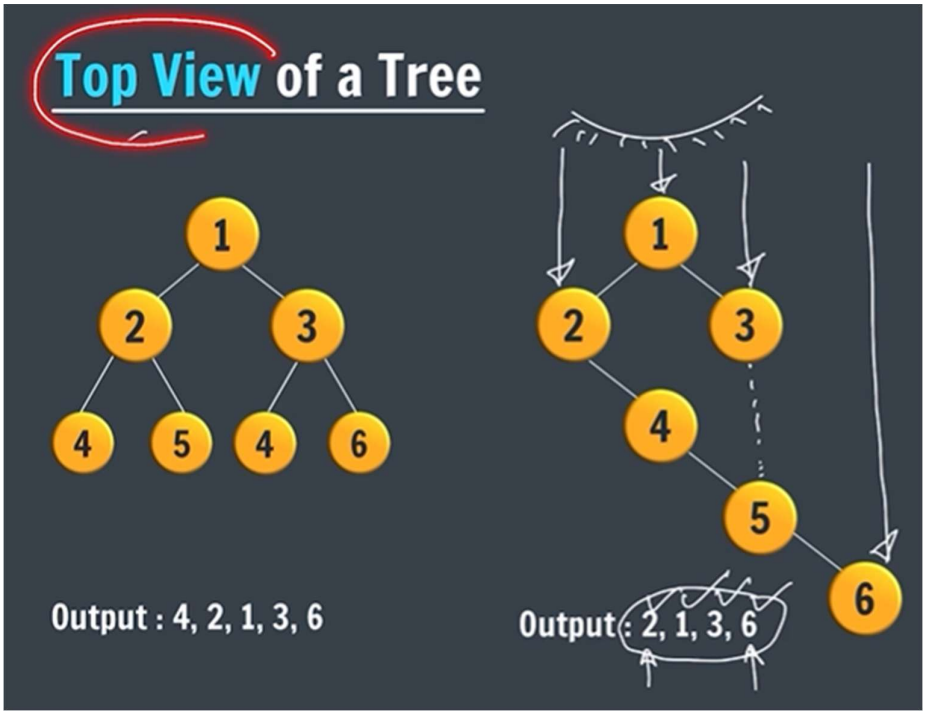

→Subtree of another Tree






→Top view of a tree

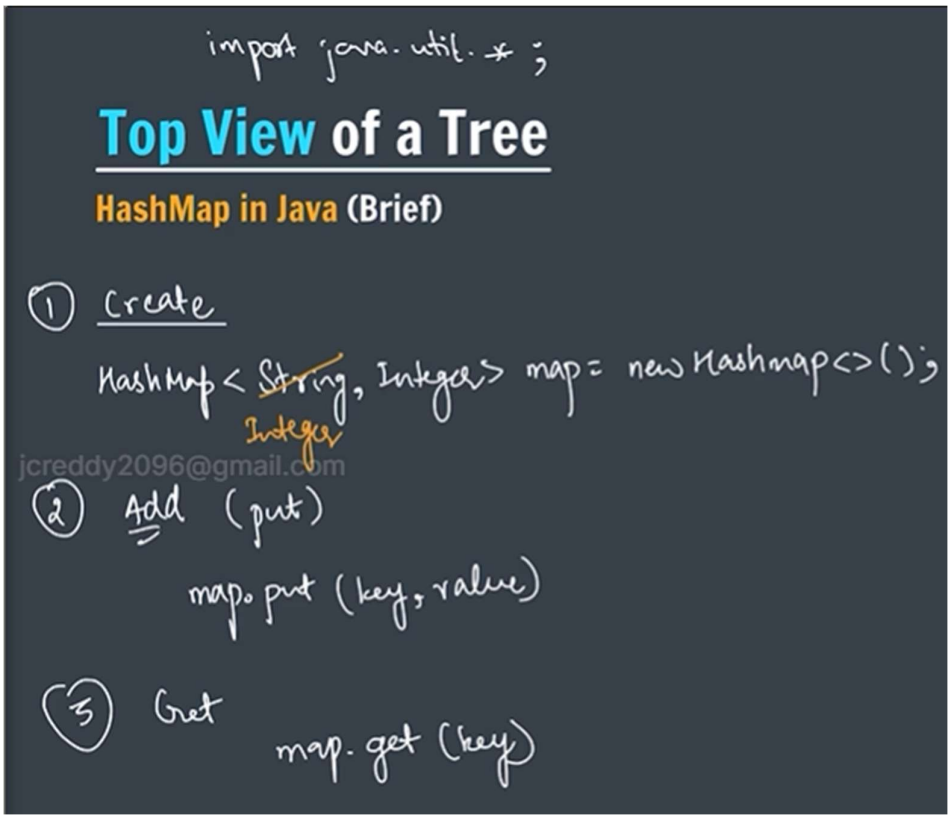



→what is map: map(key,value) key=unique value

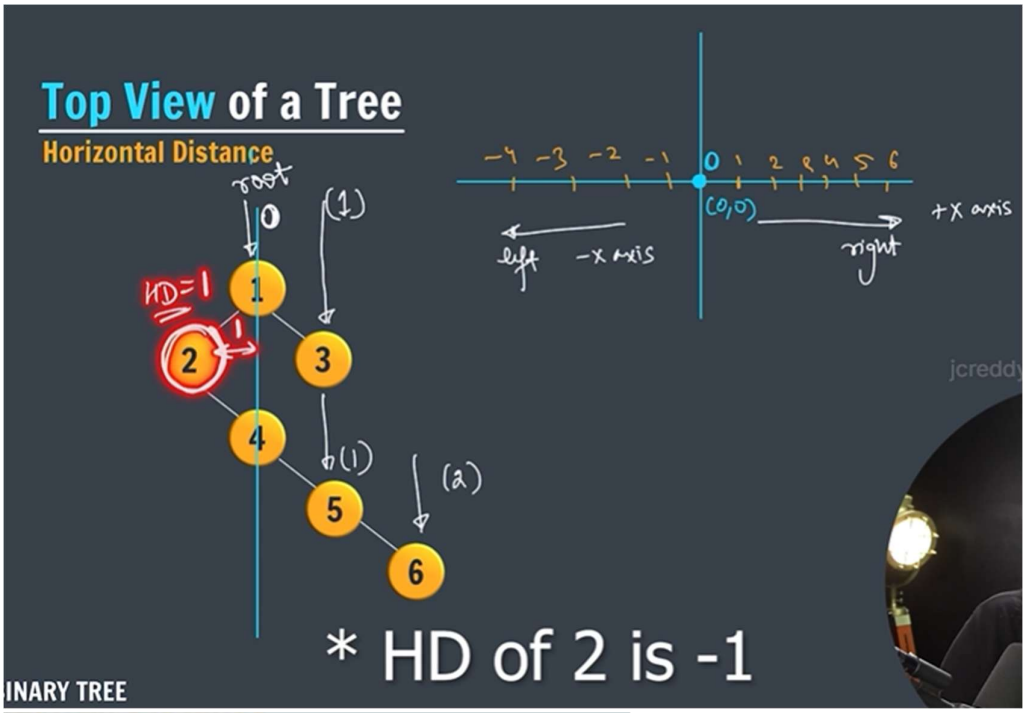

→in hashmap add –O(1)

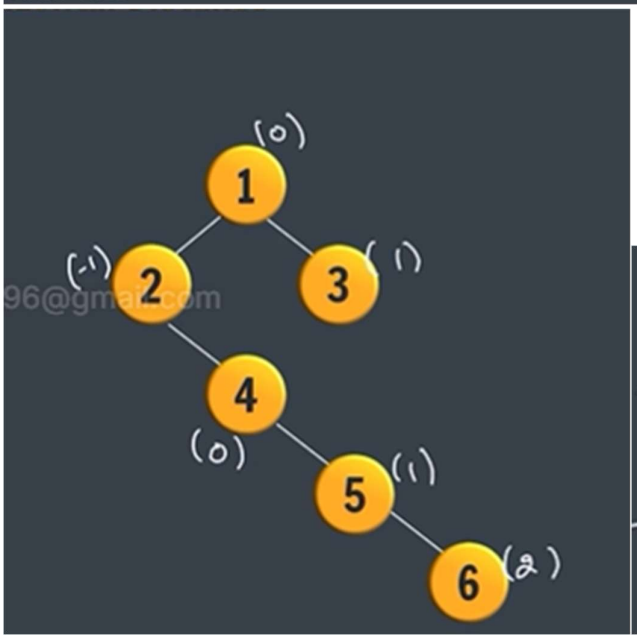
Remove –O(1)

Get – O(1)

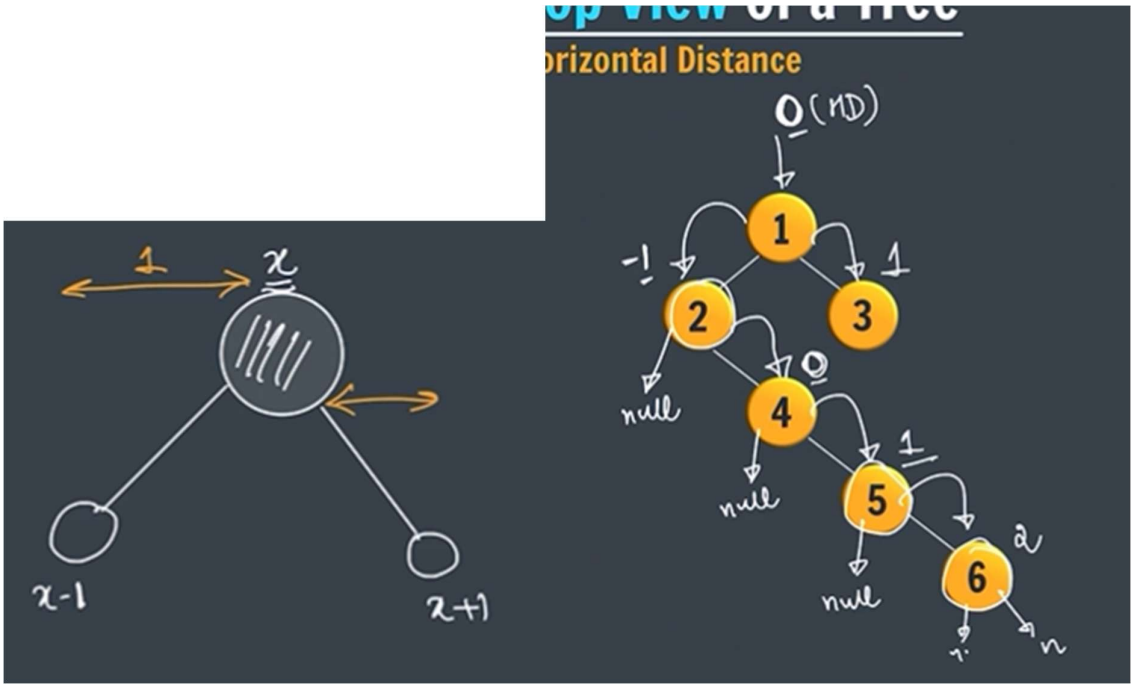



→in hash-map we need to enter the duplicate value like in hash-map it will replace the original value

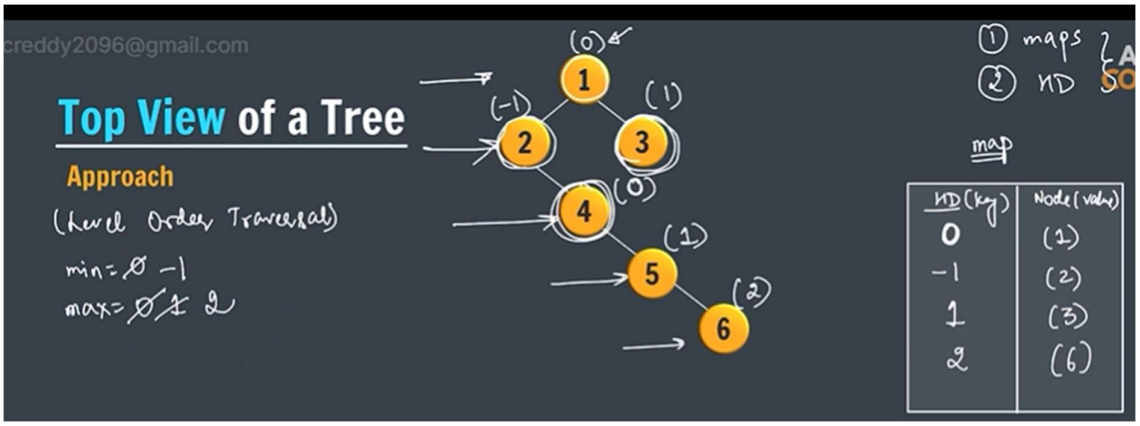


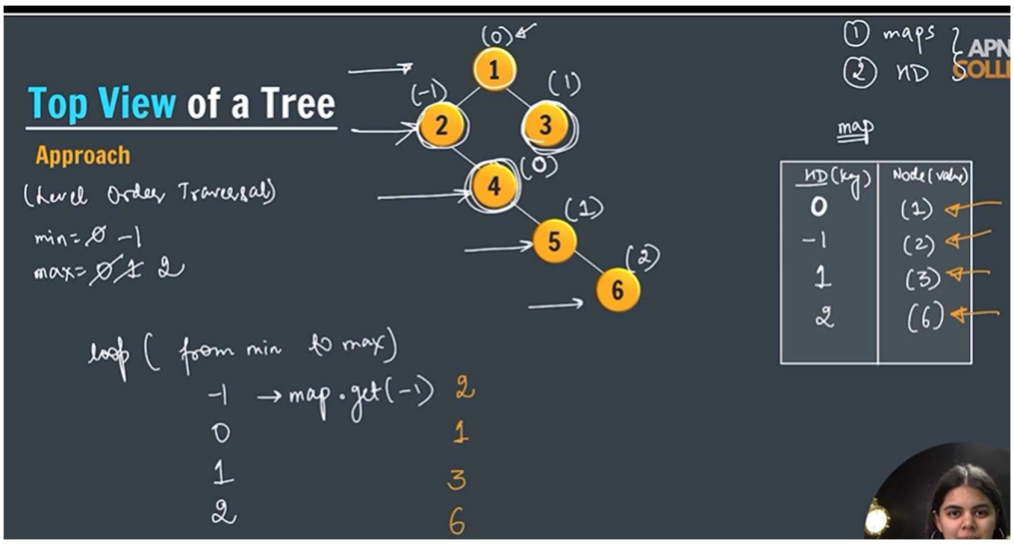




| HD | node |
|---|---|
| 0 | 1 |
| -1 | (2) |
| 1 | (3) |
| 2 | (6) |


rizontal Distance
HD
1
x
x-1
x+1
-1
1
0
1
null
null
null
2
n

→Approach


creddy2096@gmail.com
Top View of a Tree
Approach
(Level Order Traversal)
min = 0 -1
max = 0 1 2
1 maps
2 HD
map
HD (key) | Node (value)
0 | (1)
-1 | (2)
1 | (3)
2 | (6)


Top View of a Tree
Approach
(Level Order Traversal)
min = 0 -1
max = 0 1 2
1 maps
2 HD
map
HD (key) | Node (value)
0 | (1)
-1 | (2)
1 | (3)
2 | (6)
loop ( from min to max)
-1 → map.get(-1) 2
0 1
1 3
2 6